A rover that learns the fastest path

Teaching with AI (TAI)

Andrea López Incera[1], Georgios Errikos Hlapanis[2], Mirte van der Eyden[3]

[1] Departament de didàctica de la matemàtica i les ciències experimentals, Universitat Autònoma de Barcelona
andrea.lopez.incera@uab.cat

[2] 2nd Lyceum of Kos, University of the Aegean
hlapanis@2lykko.gr, hlapanis@aegean.gr

[3] Institut für Fachdidaktik, Universität Innsbruck
mirte.van-der-eyden@uibk.ac.at



universität innsbruck
UAB Universitat Autònoma de Barcelona
Co-funded by the European Union
European International School of Barcelona
2ο ΠΕΝΤΑΘΛΟ ΓΥΜΝΑΣΙΟ ΚΩ ODYSSEAS ELYTIS MORE THAN A HIGH-SCHOOL
next VIBENSHUS GYMNASIUM

Subject: Robotics, AI and programming (age 16 onwards)

Activity duration: approx. 2h25min (level 1), 3h (level 2), 3h25min (level 3), depending on the level chosen in assignment 2.

Steps:

1. Initial ideas: what is a rover, and how could it learn? (10 min)
2. Assignment 1 – Live Reinforcement Learning: be the robot yourselves (65 min)
3. Assignment 2 – Program the cutebot to learn on a grid (30 min + 40/70/100 min, depending on level)

Learning goals:

- Understand the basic ingredients of reinforcement learning (state, action, reward, memory table) through a human role-play.
- Understand what a function is and read/write simple MicroPython functions.
- Practice programming with the help of an LLM (prompt engineering) at different levels of guidance.
- Apply the above to program and observe a real robot learning a shortest-path task on a small grid.

Materials needed (per group of 2-4 students): 1 micro:bit V2 + USB cable, 1 ELECFREAKS Cutebot + battery pack, 1 laptop with access to `python.microbit.org` and an LLM (e.g. ChatGPT or Gemini), 1 printable for the environment (Section A) and 1 printable for the robot (Section B) per pair of students, 1 printable code for level 1 (Section C) per student, and a printed 3-cell grid (cells should be 13.3cm wide so the cutebot's movement is clearly visible, the printable is a separate pdf in A3) per cutebot used.

A rover that learns the fastest path

A robotics and reinforcement learning lesson with AI

The challenge: make your rover learn to move on its own, with no one telling it where to go, and find the *fastest path* to a goal. Today you will first simulate this learning process yourselves, and then make a real cutebot do it.

Initial ideas 10 min

Question 1: What do you think a rover is? What is it used for?

Solution / notes for the teacher: In the context of space and planetary research, a rover is a robotic or crewed vehicle designed to roll over and study the surface of a moon, Mars, or other celestial bodies (such as NASA's Perseverance or Curiosity).

Question 2: Have you ever seen a robotic vacuum cleaner at home? What do you think it has in common with a rover on another planet?

Solution / notes for the teacher: Guide the discussion towards: both are autonomous mobile robots that must operate without a human directly steering them at every instant. A robot vacuum does not know your house's floor plan when you buy it – it bumps into things, senses its surroundings, and gradually finds a more efficient way to cover the room. A planetary rover faces the same problem, but on a much bigger and more dangerous scale, and with a huge communication delay to Earth, so scientists on Earth cannot program what the rover must do at every step. The core idea of the lesson is: *if a robot cannot be told exactly what to do at every step, it needs a way to try, fail, and improve on its own.* That is what today's lesson is about.

Question 3: Very generally, what do you think any autonomous robot (rover, vacuum cleaner, self-driving car...) needs in order to move around and learn on its own?

Solution / notes for the teacher: Three big ingredients, which map directly onto the hardware used today: **input** (sensors, to perceive the environment, which includes perceiving positive feedback from the environment in the form of rewards, e.g. a signal that shows the robot that it has reached a goal), **processing** (a "brain" that decides what to do and learns), and **output** (motors, to act on the world). Today the micro:bit will be the brain and the cutebot will be the motor that executes the actions.

Constructing new ideas

Today's challenge is to make a robot learn, on its own, the *fastest* way to a goal. We will simplify the scenario: instead of moving on a big planet with lots of obstacles, we will consider that the world (or Environment, as we will call it from now on) is just a two-dimensional grid that consists of squared cells, and the robot can move from one cell to another performing one action. Its objective is to **find a goal cell** starting from an initial cell, and learn to perform that path as fast as possible, that is, with **the least amount of actions** (we won't consider how fast the robot moves from one cell to another).

Assignment 1: Live Reinforcement Learning

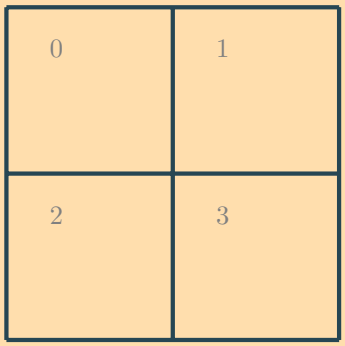
65 min

Before any robot learns, *you* will. In this assignment, you and your partner take turns

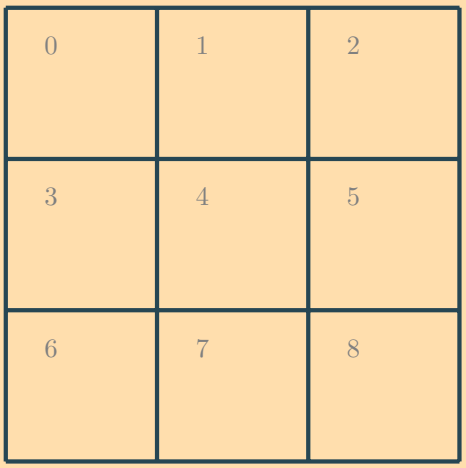
being the **Robot** and the **Environment**. The Robot has to find a hidden goal by trial and error; the Environment knows where the goal is and gives the Robot feedback.

Part A – Mental maps 25 min

- Use an **imaginary** grid on the floor (see examples below). First, use a **2×2** grid, then repeat the exercise with a **3×3** grid. Examples below show how the cells are numbered.



2×2 grid



3×3 grid

- The **Robot** starts at cell 0 (top-left) and can only call out one action at a time: "up", "down", "left" or "right". After choosing the action, the robot moves one cell in the chosen direction.
- The **Environment** secretly decides which cell holds the goal, and after every move tells the Robot only the cell number it landed on, and whether that cell is the goal.
- Play **two rounds** on each grid: **round 1** is only to find the goal (it is fine if it takes a while); in **round 2**, the Robot tries to reach the goal using the *fewest moves possible*, using whatever it remembers from round 1. In this second round, the Robot also starts at cell 0.
- If you have left over time after doing the two rounds and replying to the following questions, swap roles and repeat.

Question 4: Which grid was easier to memorize: the 2×2 or the 3×3? Why

do you think that is?

Solution / notes for the teacher: Almost always the 2×2 grid, simply because it has fewer cells and fewer possible paths to hold in memory. This sets up the next question: human short-term memory does not scale well, and a robot will need a more systematic way (a table) of remembering what it has learned, rather than "holding a picture in its head".

Question 5: Robot, what mental process did you use to find the fast path in round 2? Did you memorize exactly which action to take at each cell, or did you build a mental image (map) of the whole grid and then plan a path on it?

Solution / notes for the teacher: Answers will vary, but most students will admit to building some kind of mental *map* of the grid (knowing the overall layout, not just isolated state-action facts), and then planning the shortest route on that map, the same way you would plan a route on a floor plan of a building. This is worth highlighting explicitly: humans tend to solve this by building a map and reasoning about it globally.

Question 6: Which "mental" process do you think a robot has to do to find the shortest path? Can it create a mental map like you did or what does

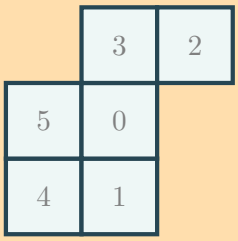
it need to store in its memory?

Solution / notes for the teacher: Even though by now there might exist robots that have the capacity of building a mental map, in Reinforcement Learning algorithms, the robot cannot do that. Thus, the robot needs to somehow store in its memory what is the best action to do for each state it perceives. In this case, the state would be the number of the cell at which it is located.

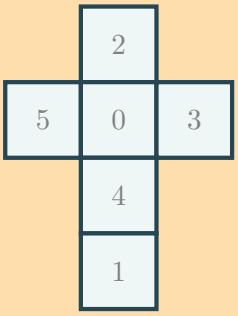
Part B – Learning without a mental map 40 min

The real robot *cannot* build a mental image of a map like you just did: it only has the limited information that each step of interaction gives it. In this part you repeat the game, but now using only the information a real robot actually has access to.

Setup: In pairs, the **Environment** designs and draws a maze of **at most 6 cells** on a gridded piece of paper (you can use the one in Section A), with a number written inside every cell. See some examples below. The Environment chooses in secret where the Robot starts and where the goal is (both can be any cell), and does **not show the maze to the Robot**.



Example of a possible maze



Another example of maze

Interaction protocol (repeat until the Robot hits the goal):

1. The Environment tells the Robot the number of the cell it is currently in (the *state*).
2. The Robot calls out one *action*: up, down, left or right; and moves accordingly.
3. The Environment tells the Robot the *reward*:
   - +10 if that move reached the goal,
   - −10 if that move tried to leave the maze or cross a wall (in which case the Robot stays in the same cell),
   - −1 otherwise (a normal move to a new, valid cell).

* One round: interactions done from the starting cell until the robot hits the goal cell.

What the Robot must do:

- On your own piece of paper (you can use the one in Section B), **draw the maze as it unfolds** while you explore it (you only find out the shape of the maze bit by bit, through your moves).
- Fill in the **memory table** provided in Section B, following the instructions there. All slots of the table start at **0**.
- From the **second round onwards**, the Robot's rule for choosing an action from a cell is: **pick the action with the highest value in your table for that cell.** If several cells still show 0 (never tried, or tied), pick randomly among them.

What the Environment must do: keep track of every reward given during the round, and at the end of the round (when the Robot reaches the goal), tell the Robot the **total reward** (the sum of all rewards given that round).

Play **several rounds** on the same maze (swap roles and do a new maze if time allows). Answer the following questions as you play (not at the end).

Question 7: After a few rounds, you may notice ties between actions in the table. Which action should the Robot choose when there is a tie?

Solution / notes for the teacher: Let students discuss this in pairs/groups before revealing the idea. Guide them towards: *look one step ahead*. For each tied action, check what cell it would lead to, and look at the **best** table value available from *that* cell. If one of the tied actions leads to a cell that already has a promising value (e.g. a +10 nearby), prefer that action. This is exactly how information about the goal "travels backwards" through the table, cell by cell, as more rounds are played – without ever having a mental map of the whole maze. Note that, depending on the shape of the maze, it may be possible that you need to look two steps ahead (not only one) to break the tie. The students can decide what to do there. For example, they can decide that when this is the case, the first action is chosen randomly among the tied ones, and then in the next step is when you look one step ahead.

Question 8: How did the Robot's total reward (given by the Environment at the end of each round) evolve over several rounds – did it increase, decrease, or stay the same? What does that tell you?

Solution / notes for the teacher: Total reward should generally *increase* (become less negative / larger) over rounds, because fewer wrong turns and wall-hits happen as the table fills in with useful values, and the Robot reaches the goal in fewer steps. This is direct, measurable evidence of learning – exactly the quantity ("reward accumulated per round") that the microbit code will track automatically in assignment 2.

Question 9: Which mechanism makes the Robot learn?

Solution / notes for the teacher: The robot learns by trial and error: after every move it overwrites its memory table with the reward it just received for that (cell, action) pair, and from then on it picks the action with the highest stored value instead of acting randomly. There is no planning or map-building — just accumulating and reusing feedback from its own past experience.

Assignment 2: Program the cutebot to learn on a grid

Preparation and initial considerations 30 min

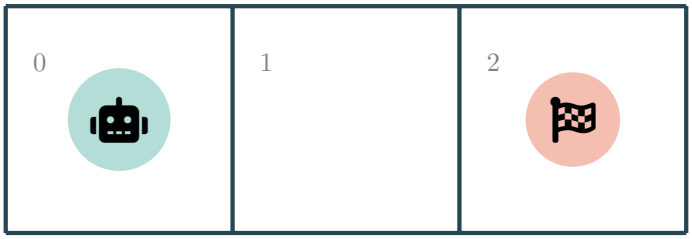
Time to make a real robot do exactly what you did by hand in assignment 1B.

Question 10: Write down, in order, the steps the Robot followed in one interaction with the environment, when you did assignment 1B. For example, after knowing the Robot starts in cell 0, the first step is that the Robot chooses an action, so you can write: 1. Choose action.

Solution / notes for the teacher:

1. Choose an action
2. Move
3. Get cell number of the cell it landed on
4. Get reward
5. Update memory table

From now on your robot will live on a small printed grid: a **3-cell** grid (see below), and it has to learn to go from cell 0 to the goal cell 2 with the least number of moves possible. Remember that the robot does not know nor see the shape of the grid (it does not know where the walls are), so it believes it can do all actions: up, down, left and right.



3-cell grid (Levels 1-3): states 0, 1, 2; goal at state 2. Print it so each cell is roughly 13.3 cm per side; state 0 is the start, facing East.

Each step you discussed in the previous question needs to be programmed so that the robot knows how to do those steps. The way to do this is to create *functions* with code, that take input information, do something with it and return an output. For example, in step 3, for the Robot to get the new cell number (output), we need to create a function that takes the information of where it was and the action it chose (inputs), computes the new cell number and ouputs this value for the robot to continue.

Function: A function is a piece of code that takes some input, does something with it, and often returns an output. We use functions when we want a process to be repeated several times, so we do not have to rewrite the same lines of code every time we need it. A function looks like this:

```
1 def double(n):
2     return 2 * n
```

Here, the input is `n`, and the output is whatever comes after `return`, in this case $2n$. Every time we need a number doubled, we can just write `double(5)` instead of writing the multiplication out again.

Question 11: a) Write your own simple function (on paper and without the help of the AI) that takes a number as input and returns that number plus one. b) How would you call that function if you needed to add one to the number 7? c) Imagine you now need to use the output of your function

from b) somewhere else and several times. How would you create a variable that stores this output?

Solution / notes for the teacher: a) e.g. `def add_one(n): return n + 1`. b) To call it, write `add_one(7)`, which returns `8`. c) We can store the output `8` in the variable `s=add_one(7)`, which will then be `s=8`. The key point is that students correctly identify the input (what goes in the parentheses) and the output (what comes after `return`) in the definition; and how to use the function in the code (call it and use the output later with a variable).

Try it on the micro:bit: Connect your micro:bit to the laptop with the USB cable (the cutebot won't do anything yet), open `python.microbit.org`, and do the following tasks.
1. Open a new file and write your function for adding plus one at the beginning of the code. So far, the output won't be shown anywhere.
2. Make the LED display of the micro:bit show the output of your function for the input '7'. You can ask your AI for help with the following prompt:

"Write a simple MicroPython program for a micro:bit that shows the result of my function `add_one(7)` on the LED display."

Paste the code into the editor, click **Send to micro:bit**, and check that it works! Can you find the function being used to show the image? What are the input and output?

Programming the learning cutebot 40/70/100 min (depending on level)

You will work with micro:bits and cutebots. The micro:bit will be the brain of the robot, and the ELECFREAKS Cutebot will be the motor that will move it across the grid with its two independent wheels.
The micro:bit will control the entire interaction environment-robot: it will contain and update the robot's memory (the state-action table), it will give the order of "moving" to the cutebot and it will process what is happening in the environment (e.g. if the robot finds the goal or not, or if it hits a wall) to give feedback to the robot. You will program such interaction in the micro:bit as you did above for your function, but this time, the code will be longer.

Things to consider:

1. You will program what you did in assignment 1B now in the cutebot.

2. The reward scheme is exactly the one you used in assignment 1: -1 for a normal move, -10 for hitting the edge of the grid, $+10$ for reaching the goal.

3. The robot will have a memory that is exactly the state-action table you filled in in the previous assignment. It also updates the memory table as you did: it overwrittes the values with the rewards given by the environment.

4. The starting cell of the cutebot is always **cell 0** on your printed grid, and you should place the cutebot facing East (right).

Now choose one out of three levels of guidance, and follow the instructions on that section. Make sure to prepare the cutebot before starting programming (box below).

1. Level 1: AI did the programming, the code is given, and you just see it running and try to use your intuition to find certain functions in the code.

2. Level 2: You prompt the AI to generate the code in one go, taking into account the steps you did in assignment 1B. You then see it running and identify certain functions in the code.

3. Level 3: You are in charge of programming all the steps, for which you can use AI for help.

Prepare the cutebot: 10 min

- Slot the micro:bit into the top of the cutebot and make sure the battery pack is connected and switched on.
- Open `python.microbit.org`
- (Step only needed if you are doing levels 2 or 3) Make sure you add the library `Cutebot.py` (it comes with this handout as student material), which contains all the code for the robot wheel movements, etc, to the python editor. In the tab "Project" on the left, you should have a `main.py` file where you paste or create the code for the robot and another file `Cutebot.py` loaded. To load this last file, click on 'Open', then select your file, then before clicking on 'Confirm', press the settings icon to the side of the file name and select 'Add Cutebot.py' (this is to avoid replacing the main.py file with the Cutebot.py file), then 'Confirm'.

In any level, whenever you use AI for programming, make sure you take into account the following prompting rule.

The golden prompting rule: Always start your prompt with *"I am programming an ELECFREAKS Cutebot with a micro:bit, using MicroPython and the* `Cutebot` *library."* This helps the AI use the correct commands. Also give the AI the file `Cutebot.py` (already loaded in your editor) so it knows which functions are available, e.g. `ct.set_motors_speed(left_speed, right_speed)`, with speeds from -100 to 100.

Level 1

30 min

In this level, the AI did the programming already, and you can find the code in Section C printed. Before seeing it run, try and find some functions in the code. For that, use your intuition and read the comments made by the AI programmer (in grey, with a # in front), which are there to explain what the following lines of code do.

On your printed copy of the code, find and circle the following:

- ☐ The line where the memory table is created, and what value every slot starts at.
- ☐ The function that computes which cell an action leads to (step 3).
- ☐ The line(s) inside that same function where the reward +10 is given if the robot reaches the goal, and the line where −10 is given if it hits the edge of the maze.
- ☐ Inside the function `interaction(state)`, can you identify which line of code corresponds to each of the steps that you discussed before? Note: the input `state` is the current state of the robot (cell number), right before it moves or chooses an action.

Solution / notes for the teacher:

- Table creation: `table = [[0, 0, 0, 0] for _ in range(3)]` – every slot starts at 0.
- Geometry only: `def next_state(state, action)` increases or decreases the state by one for `EAST`/`WEST` (checking it does not go past cell 0 or cell `GOAL`), and simply reports an edge hit for `NORTH`/`SOUTH` (there is no up/down move on this corridor), returning the new state together with the reward.
- +10: `return (ns, 10) if ns == GOAL else (ns, -1)` (reward is given for reaching the goal). −10: `return state, -10` whenever the move would leave the corridor, or whenever the action is `NORTH` or `SOUTH`.

  Inside the function `interaction(state)`:
- Action choice: `action = choose_action(state)`
- Get new cell number (`ns`) and reward (`reward`) considering the previous state and the chosen action: `ns, reward = next_state(state, action)`
- Move (once it knows whether it hit the wall or not): `rotate_to(action)` rotates the robot to face the chosen direction, and `if ns != state: forward()` makes a move forward only if the new state is different from the previous one (i.e. if it did not hit a wall with its decision)
- Update the memory table with the reward: `table[state][action] = reward`. The code after that just shows a heart in the display if it reached the goal.

▶ **Now flash it!** Copy the code printed in Section C (also provided as `RL_cutebot_3cell.py` in the Student Materials, already calibrated for ≈13.3 cm cells) into `python.microbit.org`, send it to your micro:bit, place the robot in cell 0 facing east, and press **button A** to run one episode. Run it several times in a row and watch the total reward, scrolled at the end of each episode, improve. You should also see how the robot does the path to the goal in less movements each time (at the beginning it may move back and forth or hit the walls a lot).

Short on time? The `HEX_files` folder in the Student Materials contains ready-to-flash `.hex` files for every piece of code in this handout (`RL_cutebot_3cell.hex`, `RL_cutebot_4cell.hex`). Just connect the micro:bit by USB (it appears as a `MICROBIT` drive) and drag the `RL_cutebot_3cell.hex` file onto it – no editor, no AI, no copy-pasting of code lines needed.

Level 2

60 min

In this level, your goal is to prompt the AI to get the code to make the robot learn, based on exactly what you did in assignment 1. Complete the prompt below before sending it to the AI. The technical details of the hardware are already given at the beginning, as well as what the micro:bit has to display for you to check that the robot is doing what it has to do.

- Read the prompt fully to make sure you know what you are asking the LLM to do.
- Input a description of each part of the process you did in assignment 1, as detailed as possible, in the corresponding space in the prompt below and then send it to the AI.
- Paste the AI's output in the Python editor and send it to your micro:bit.
- Place the cutebot in cell 0 facing east, and press **button A** to run one episode.
- Run it several times in a row and observe what the robot does.
- Check with the check list below if the robot is actually learning. If not, re-prompt it and try again.

Replace each bracket space "[ADD YOUR DESCRIPTION HERE]" with your own description, then send this prompt to your AI:

"I am programming an ELECFREAKS Cutebot with a micro:bit, using MicroPython and the Cutebot library (Cutebot.py). SPEED = 24, MOVE_TIME = 600 ms, TURN_TIME = 300 ms (one grid cell takes MOVE_TIME ms to cross, and a 90-degree turn takes TURN_TIME ms). Track a heading variable (North/East/South/West) so the cutebot always turns the correct way before driving forward.

Robot: [ADD YOUR DESCRIPTION HERE]

Environment: [ADD YOUR DESCRIPTION HERE]

Reward: [ADD YOUR DESCRIPTION HERE]

Memory table: [ADD YOUR DESCRIPTION HERE]

Interaction: [ADD YOUR DESCRIPTION HERE]

At each step, before moving, show an arrow on the LED display that matches the chosen action (e.g. `Image.ARROW_N/E/S/W`*).*

Run one episode when button A is pressed, show a heart on the LEDs for a positive reward, and scroll the total reward accumulated during the episode once the goal is reached."

Solution / notes for the teacher: Expected fill-ins descriptions: robot: describe the possible actions and what each one does; environment: describe the grid; rewards: -10 (hit a wall)/ $+10$ (got to the goal state) / -1 (any other case); memory table: every slot starts at 0, "overwrite the table slot for the (state, action) just taken with the reward just received, replacing the old value completely"; "pick the action whose table value is currently highest for that cell"; "choose randomly among the tied actions". Check that the LLM's code matches the code in Section C in spirit (same reward scheme, same overwrite rule, same random tie-break) – if the LLM instead produces the standard α/γ Q-learning update, ask students to re-prompt asking explicitly for "no learning rate, just overwrite".

Test that the generated code makes the robot learn by checking the following things:

- ☐ Did the action symbol shown on the display match the actual movement the cutebot did?
- ☐ Is the final reward shown on the display increasing as you run more episodes?
- ☐ Do you see a change in the behaviour of the robot as you run more episodes? Does it seem to reach the goal in less steps each time?

If any of the answers above is 'No', check the code and discuss with your partner *which part of the prompt* was ambiguous, refine it and run the new AI-generated code. In Section C, you can find an example code that works. Use it as a reference for comparison to try and find the things that don't work.
If all the answers are 'Yes', it means that the robot is learning! Then, you can have a look at the code and try to find the following functions. For that, use your intuition

and read the comments made by the AI programmer (in grey, with a # in front), which are there to explain what the next lines of code do.

On the AI-generated code, find and circle the following:

- ☐ The line where the memory table is created, and what value every slot starts at.
- ☐ The function that computes which cell an action leads to.
- ☐ The line(s) inside that same function where the reward +10 is given if the robot reaches the goal, and the line where −10 is given if it hits the edge of the maze.
- ☐ Inside the function that describes one interaction of the robot with the environment, can you identify which line of code corresponds to each of the steps that you discussed before in question 10? Note: normally, the function for the interaction will input `state`, which is the current state of the robot (cell number), right before it moves or chooses an action.

Level 3

90 min

Build the whole program yourself, step by step, with the LLM helping you at each individual step rather than writing all the code at once. If you have some experience with coding, you can also try and do it yourself without the AI.

☐ **Step 1 – Move and turn.** Ask the AI for two separate functions: one that drives the cutebot forward exactly one cell, and one that turns it 90 degrees (without forward movement). Use the calibration values from Section E (`SPEED`, `MOVE_TIME`, `TURN_TIME`), which are calibrated on a grid with squared cells of size 13,3 cm. Test both on the grid before moving on.

☐ **Step 2 – Create the memory table.** Ask the AI for a table (a list of lists) with one row per cell and one column per action, every slot starting at 0. Print it to check it has the right shape.

☐ **Step 3 – One interaction, no learning yet.** Ask the AI for code that: reads the current state, picks *any* valid action (random is fine for now), and physically moves the cutebot there (turn, then drive). If the cutebot picks an action that makes it hit a wall, it should stay still where it was. Do not add reward or table updates yet – just get one single move working end to end. Check that it works fine for all four actions.

☐ **Step 4 – Add the reward and update the table.** Now ask the AI to compute the reward for the move you just made ($-1/-10/+10$, as in assignment 1), and to **overwrite** the table slot for the (state, action) just taken with that reward.

☐ **Step 5 – Choose the best action.** Ask the AI to change the action-choice rule so that, instead of picking randomly, it picks the action with the highest table value for the current cell, breaking ties randomly.

☐ **Step 6 – Loop the entire interaction.** Ask the AI to repeat Steps 3-5 in a loop until the goal is reached (one full episode). Track and display the total reward for the episode.

Question 12: At which step did your cutebot first visibly "do something different" because of what it had learned, rather than acting randomly? Why only from that step onwards?

Solution / notes for the teacher: Only from Step 5 onwards, because that is the first point where the action-choice rule actually looks at the table instead of picking randomly – Steps 1-4 build all the necessary pieces (movement, memory, one interaction, reward/update) but none of them yet changes *behaviour* based on what was learned. This is a good moment to stress that "learning" only becomes visible once the decision rule is coupled back to the stored memory.

The complete reference solution for Level 3 (identical in structure to the one given directly in Level 1, but built up through the five steps above) is in Section C.

Only for those who want a bigger challenge: on the 2×2 grid from assignment 1, genuine ties between actions can appear (more than one direction can lead toward the goal), so picking randomly is no longer good enough – the Robot needs to "look one step ahead", exactly as you discussed in assignment 1. Section D gives the advanced 4-cell version of the code, with that one-step lookahead tie-break added back in. Try building it yourself first, using the same five steps, before checking the appendix.

Question 13: Run several episodes in a row (table values are kept between episodes). Does the total reward shown at the end of each episode improve? Compare this to what you observed by hand in assignment 1.

Solution / notes for the teacher: Yes: early episodes take many moves (lots of -1s and the occasional -10 near the edges), so the total reward is a large negative number; later episodes reach the goal in close to the minimum number of moves, so the total reward gets close to its best possible value. This is exactly the same improvement students observed by hand with their paper table in assignment 1 – the cutebot is doing, mechanically and much faster, precisely what they did themselves.

What have we learned?

Individually, write a short reflection:

- What is reinforcement learning, in your own words, and how did your robot use it to find the fastest path?
- What was the hardest part of "being the robot" in assignment 1, and how does

that compare with what the cutebot's code actually has to do?

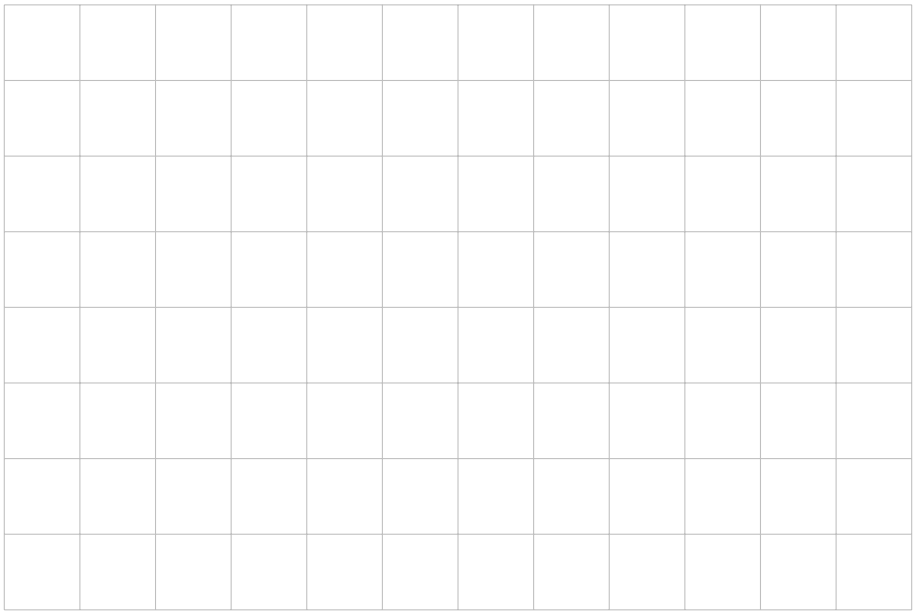
- Where did you use AI today, and how did it help (or not help) you?

Question 14: Write your answers here:

A Environment (assignment 1B)

Design your own grid of max. 6 cells below, and **don't show it to the Robot!** It can have any shape you want, as long as each cell is connected to at least one other cell.

Draw your maze here. Don't forget to mark the initial cell, the goal cell and to write the cell numbers (from 0 to max. 5). The cell numbers don't have to follow any order. If your maze has less than 6 cells, you have to tell the Robot which cell numbers (from 0 to 5) are not used.



B Robot (assignment 1B)

Fill in the table (with pencil!) as you interact with the environment.

1. Start by adding 0 in all table slots.
2. After each move, erase the current value of the state-action slot you have done, and add instead the reward that the environment gives you once you move. For example, if you were in cell 0 and you move up, you have to fill in the top left slot.
3. From the **second round onwards**, the Robot's rule for choosing an action from a cell is: **pick the action with the highest value in your table for that cell.** If several cells still show 0 (never tried, or tied), pick randomly among them.

This sheet is for one maze (if the maze has less than 6 cells, cross out the non-valid rows – the environment should tell you which ones to cross).

Actions / **States**	**Up**	**Down**	**Left**	**Right**
Cell 0				
Cell 1				
Cell 2				
Cell 3				
Cell 4				
Cell 5				

Blank paper for the Robot to draw the maze as it unfolds:



C Code listing for the learning cutebot (3-cell grid)

The full reference code for assignment 2 (Levels 1-3, 3-cell grid) is given below. It is also provided as `RL_cutebot_3cell.py` in the Student Materials, already calibrated for ≈13.3 cm cells.

Listing 1: RL_cutebot_final.py – final code for assignment 2 (3-cell grid), flashed as RL_cutebot_3cell.hex

```
from microbit import *
from Cutebot import *
import random

ct = CUTEBOT()

# --- calibration parameters to define speed and turning time of the robot ---
SPEED = 24
MOVE_TIME = 600      # ms per grid cell (~13.3 cm cells)
TURN_TIME = 300      # ms per 90-degree turn

# --- grid definition: a straight line of 3 cells, numbered 0, 1, 2 ---
GOAL = 2

NORTH, EAST, SOUTH, WEST = 0, 1, 2, 3
ACTIONS = [NORTH, EAST, SOUTH, WEST]
heading = EAST

# basic movements
def forward():
    ct.set_motors_speed(SPEED, SPEED)
    sleep(MOVE_TIME)
    ct.set_motors_speed(0, 0)

def turn_left():
    ct.set_motors_speed(-SPEED, SPEED)
    sleep(TURN_TIME)
    ct.set_motors_speed(0, 0)

def turn_right():
    ct.set_motors_speed(SPEED, -SPEED)
    sleep(TURN_TIME)
    ct.set_motors_speed(0, 0)

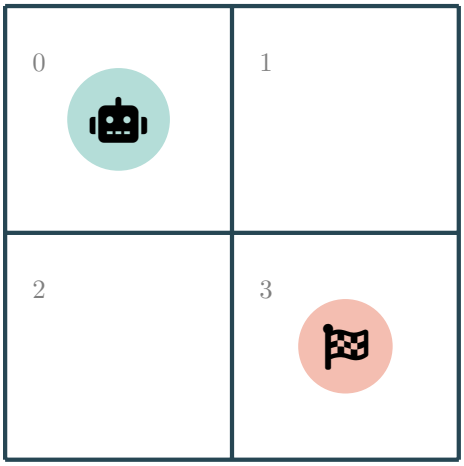
def rotate_to(target):
    global heading
    diff = (target - heading) % 4
    if diff == 1:   turn_right()
    elif diff == 2: turn_right(); sleep(200); turn_right()
    elif diff == 3: turn_left()
    heading = target
```

```
42
43  # the memory table: one row per cell, one column per action --
        all slots start at 0
44  table = [[0, 0, 0, 0] for _ in range(3)]
45
46  # which cell does "action" lead to from "state"? (also carries
        the reward)
47  def next_state(state, action):
48      if action == EAST:
49          if state + 1 > GOAL:
50              return state, -10       # hit the edge, stays in place
51          ns = state + 1
52      elif action == WEST:
53          if state - 1 < 0:
54              return state, -10       # hit the edge, stays in place
55          ns = state - 1
56      else:
57          return state, -10           # NORTH/SOUTH: no such move in
                this corridor
58      return (ns, 10) if ns == GOAL else (ns, -1)
59
60  # choose an action using the table (ties broken randomly)
61  def choose_action(state):
62      best_value = max(table[state])
63      candidates = [a for a in ACTIONS if table[state][a] ==
            best_value]
64      return random.choice(candidates)
65
66  # one interaction: choose an action, move, get the reward, update
        the table
67  def interaction(state):
68      action = choose_action(state)
69      ns, reward = next_state(state, action)
70      rotate_to(action)
71      if ns != state:
72          forward()
73      table[state][action] = reward     # overwrite the current
            values with reward
74      if reward == 10:
75          display.show(Image.HEART)
76      return ns, reward
77
78  # loop over a full episode
79  def run_episode():
80      global heading
81      heading = EAST
82      state = 0
83      total_reward = 0
84      while state != GOAL:
85          state, reward = interaction(state)
86          total_reward += reward
```

```
87          display.show(str(state))
88          sleep(1500)
89      display.show(Image.HAPPY)
90      sleep(300)
91      display.scroll(str(total_reward))
92
93  display.scroll("A")
94  while True:
95      if button_a.was_pressed():
96          run_episode()
```

D Advanced challenge: the 2×2 grid

This appendix is only for students who choose the advanced route in assignment 2, Level 3 (see the challenge box there). The robot now moves on a 2×2 grid (4 cells), where genuine ties between actions can appear, so the Robot must look one step ahead to break them, as discussed in assignment 1.



4-cell grid (Level 3, advanced): states 0-1-2-3, goal at state 3. Print it so each cell is roughly 13.3 cm per side; state 0 is the start, facing East.

Listing 2: RL_cutebot_final_2x2.py – advanced code for the 2×2 grid, with one-step lookahead tie-breaking, flashed as RL_cutebot_4cell.hex

```
from microbit import *
from Cutebot import *
import random

ct = CUTEBOT()

# --- calibration, from Assignment: Calibrate your rover ---
SPEED = 24
MOVE_TIME = 600       # ms per grid cell (~13.3 cm cells)
TURN_TIME = 300       # ms per 90-degree turn

# --- grid definition: the 2x2 grid ---
ROWS, COLS = 2, 2
GOAL = ROWS * COLS - 1

NORTH, EAST, SOUTH, WEST = 0, 1, 2, 3
ACTIONS = [NORTH, EAST, SOUTH, WEST]
heading = EAST

# ===== STEP 1: basic movements =====
def forward():
    ct.set_motors_speed(SPEED, SPEED)
    sleep(MOVE_TIME)
    ct.set_motors_speed(0, 0)

```

```
def turn_left():
    ct.set_motors_speed(-SPEED, SPEED)
    sleep(TURN_TIME)
    ct.set_motors_speed(0, 0)

def turn_right():
    ct.set_motors_speed(SPEED, -SPEED)
    sleep(TURN_TIME)
    ct.set_motors_speed(0, 0)

def rotate_to(target):
    global heading
    diff = (target - heading) % 4
    if diff == 1:   turn_right()
    elif diff == 2: turn_right(); sleep(200); turn_right()
    elif diff == 3: turn_left()
    heading = target

# ===== STEP 2: the memory table =====
# one row per cell, one column per action -- all slots start at 0
table = [[0, 0, 0, 0] for _ in range(ROWS * COLS)]

# geometry: which cell does "action" lead to from "state"? (also carries the reward)
def next_state(state, action):
    r, c = state // COLS, state % COLS
    if action == NORTH: r -= 1
    elif action == EAST: c += 1
    elif action == SOUTH: r += 1
    elif action == WEST: c -= 1
    if r < 0 or r >= ROWS or c < 0 or c >= COLS:
        return state, -10          # hit the edge, stays in place
    ns = r * COLS + c
    return (ns, 10) if ns == GOAL else (ns, -1)

# ===== STEP 5: choose an action using the table (with one-step lookahead tie-break) =====
def choose_action(state):
    best_value = max(table[state])
    candidates = [a for a in ACTIONS if table[state][a] == best_value]
    if len(candidates) == 1:
        return candidates[0]
    best_lookahead = None
    best_actions = []
    for a in candidates:
        ns, _ = next_state(state, a)
        lookahead = max(table[ns])
        if best_lookahead is None or lookahead > best_lookahead:
            best_lookahead = lookahead
            best_actions = [a]
```

```
        elif lookahead == best_lookahead:
            best_actions.append(a)
    return random.choice(best_actions)

# ===== STEP 3 + STEP 4: one interaction, with reward and table update =====
def step(state):
    action = choose_action(state)
    ns, reward = next_state(state, action)
    rotate_to(action)
    if ns != state:
        forward()
    table[state][action] = reward     # overwrite, no learning rate
    if reward == 10:
        display.show(Image.HEART)
    return ns, reward

# ===== STEP 6: loop over a full episode =====
def run_episode():
    global heading
    heading = EAST
    state = 0
    total_reward = 0
    while state != GOAL:
        state, reward = step(state)
        total_reward += reward
        display.show(str(state))
        sleep(1500)
    display.show(Image.HAPPY)
    sleep(300)
    display.scroll(str(total_reward))

display.scroll("A")
while True:
    if button_a.was_pressed():
        run_episode()
```

E Calibration reminder

This handout assumes your rover has already been calibrated (e.g. in an earlier session), with the functions `forward()` and `turn_right()`/`turn_left()` driving it **exactly one grid cell** and **exactly 90 degrees**, respectively. The code in listing 1 uses `SPEED = 24`, `MOVE_TIME = 600` ms and `TURN_TIME = 300` ms, calibrated for grid cells of about 13.3 cm; re-calibrate these three numbers for your own rover and floor if the cutebot drifts off the grid (see the `HEX_files/Calibration` folder in the Student Materials for minimal standalone scripts testing each of the three numbers separately, both as `.py` files and pre-flashed as `01_forward.hex`, `02_turn90.hex` and `03_square.hex`).
In general, cutebots are not super precise, so it may be difficult to calibrate it properly, which means that especially in the first episodes of the learning process, the robot may get out of the grid when it turns around.

© 2026 “Teaching in AI” (TAI) Consortium.

This educational material was developed within the framework of the Erasmus+ project “Teaching in AI” (TAI) (2025-1-DK01-KA226-SCH-000365972).

Funded by the European Union.

The views and opinions expressed are those of the author(s) only and do not necessarily reflect those of the European Union or the granting authority. Neither the European Union nor the granting authority can be held responsible for them.

This material is licensed under the Creative Commons Attribution 4.0 International Licence (CC BY 4.0), unless otherwise indicated.